

Universidad del Valle de Guatemala

Programación Orientada a Objetos

Segundo Ciclo, 2026

Catedrático: Kevin Antonio Velasquez Aguilar



Laboratorio 2 - Arreglos y excepciones
AGENCIA DE DETECTIVES – CASO MISTERIOSO

Obed Natanael Tot Paiz
Carné:26971

Fecha de entrega: 30/9/2026

Análisis

Responda las siguientes preguntas:

1. ¿Qué propiedades y métodos tendrá cada clase?

Clase	Propiedades	Métodos
Ubicación	Código, nombre, dirección, nivel de riesgo y estado.	Constructor, métodos para consultar sus propiedades, actualizar () para modificar riesgo y estado, y toString() para representar su información.
Pista	Código, descripción, tipo de evidencia, nivel de importancia y nivel de confiabilidad.	Constructor, métodos para consultar sus propiedades, actualizar () para modificar su información y toString().
Caso	Nombre, código, detective responsable, arreglo de ubicaciones y colección de pistas.	Constructor; métodos para registrar, consultar, buscar, modificar y eliminar registros; métodos para contar ubicaciones y pistas, calcular espacios disponibles, obtener máximos y calcular el promedio de importancia.
Principal	Un Scanner llamado entrada y una referencia al casoActual.	main(), mostrarMenu(), ejecutarOpcion(), crearCaso(), leerTexto(), leerEntero() y mostrarReporte().

La clase Caso administrará los registros, mientras que Principal manejará la interacción con el usuario.

2. ¿Qué tipo deben tener las propiedades y métodos de cada clase?

Clase	Propiedad	Tipo
Ubicación	codigo, nombre, direccion, estado	String
Ubicación	nivelRiesgo	int
Pista	codigo, descripcion, tipoEvidencia	String
Pista	nivelImportancia, nivelConfiabilidad	int
Caso	nombre, codigo, detectiveResponsable	String
Caso	ubicaciones	Ubicacion[]
Caso	pistas	ArrayList<Pista>

Principal	entrada	Scanner
Principal	casoActual	Caso

Los métodos tendrán los siguientes tipos de retorno

- Los métodos de consulta de propiedades devolverán el tipo correspondiente a cada atributo.
- Los métodos para registrar, modificar y eliminar serán void.
- Los métodos de búsqueda devolverán Ubicacion o Pista, según corresponda.
- Los métodos para contar registros y espacios disponibles devolverán int.
- Los métodos para obtener el mayor riesgo, importancia o confiabilidad devolverán el objeto Ubicacion o Pista seleccionado.
- El promedio de importancia será de tipo double.
- Los métodos toString() y las consultas generales que preparen información para mostrar devolverán String.
- crearCaso() devolverá Caso, leerTexto() devolverá String y leerEntero() devolverá int.
- ejecutarOpcion() devolverá boolean para indicar si el menú debe continuar.
- main(), mostrarMenu() y mostrarReporte() serán void.
- Los constructores no tienen tipo de retorno.

3. ¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo básico?

¿Qué tipo de objetos almacenará y cuál será su tamaño?

La propiedad ubicaciones de la clase Caso debe implementarse utilizando un arreglo básico de tipo Ubicacion[]. Almacenará objetos de la clase Ubicacion y tendrá un tamaño fijo de cinco posiciones. Las posiciones que no tengan una ubicación registrada contendrán null.

4. ¿Cuál de las propiedades identificadas debe implementarse utilizando un ArrayList? ¿Qué tipo de objetos almacenará?

La propiedad pistas de la clase Caso será de tipo ArrayList<Pista>. Almacenará objetos de la clase Pista y permitirá agregar o eliminar elementos durante la investigación, porque la cantidad de pistas no se conoce desde el inicio y se utilizará ArrayList directamente, sin declarar la colección como List.

5. ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?

Todos los atributos serán private para proteger los datos y evitar que otras clases los modifiquen directamente. Los constructores y los métodos que otras clases necesiten utilizar serán public, como los métodos de consulta, registro, modificación y eliminación.

Los métodos auxiliares que solo se utilicen dentro de una clase serán private. Por ejemplo, validarPosicion() será privado en Caso.

El main() será public static. Los atributos y los demás métodos de apoyo serán private static, porque se utilizarán desde el contexto estático del programa principal.

6. ¿Qué parámetros serán requeridos por los métodos en sus clases?

Método u operación	Parámetros necesarios
Constructor de Ubicacion	String codigo, String nombre, String direccion, int nivelRiesgo, String estado.
Constructor de Pista	String codigo, String descripcion, String tipoEvidencia, int nivelImportancia, int nivelConfiabilidad.
Constructor de Caso	String nombre, String codigo, String detectiveResponsable.
Registrar ubicación	int posicion, Ubicacion ubicacion.
Buscar o descartar ubicación	int posicion.
Modificar ubicación desde Caso	int posicion, int nivelRiesgo, String estado.
Actualizar un objeto Ubicacion	int nivelRiesgo, String estado.
Registrar pista	Pista pista.
Buscar o eliminar pista	String codigo.
Modificar pista desde Caso	String codigo, String descripcion, String tipoEvidencia, int nivelImportancia, int nivelConfiabilidad.
Actualizar un objeto Pista	Descripción, tipo de evidencia, importancia y confiabilidad, con los tipos anteriores.
Validar posición	int posicion.
Leer texto o entero	String mensaje.
Ejecutar opción del menú	int opcion.
main()	String[] args.

Los métodos para consultar propiedades, mostrar el menú, crear un caso desde la entrada del usuario y realizar cálculos no necesitarán parámetros en esta propuesta, el código de una pista se conservará como identificador durante las modificaciones.

7. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores deberán validarse antes de modificar el estado de los objetos?

Los valores iniciales se proporcionarán mediante los constructores, utilizando la información ingresada por el usuario.

Al crear un caso, su constructor inicializará el arreglo de cinco ubicaciones, cuyas posiciones comenzarán en null, y un ArrayList<Pista> vacío. Antes de crear o modificar objetos se va a tener que hacer:

- Los textos obligatorios no sean null, vacíos ni contengan únicamente espacios.
- El nivel de riesgo esté entre 1 y 10.
- El nivel de importancia esté entre 1 y 10.
- El nivel de confiabilidad esté entre 0 y 100.
- La posición del arreglo esté entre 0 y 4.
- La posición esté disponible al registrar una ubicación.
- No exista otra pista con el mismo código al registrarla.
- El registro exista antes de modificarlo o eliminarlo.
- Se validarán todos los nuevos datos antes de asignarlos, para evitar modificaciones incompletas cuando alguno sea incorrecto.

8. ¿Cómo determinará si una posición del arreglo contiene una ubicación o contiene null?

Primero se comprobará que la posición esté dentro de los límites del arreglo. Después se evaluará la condición `ubicaciones[posicion] != null`. Y si la condición es verdadera, existe una ubicación y pueden utilizarse sus métodos. Si es falsa, la posición está vacía y esta verificación se realizará tanto en consultas individuales como durante los recorridos del arreglo.

9. ¿Cómo realizará las operaciones de búsqueda, modificación y eliminación dentro del ArrayList?

Para la búsqueda, se recorrerá el ArrayList y se comparará el código de cada pista con el solicitado utilizando `equals()`. Se devolverá la pista encontrada o null si no existe. Para la modificación, primero se buscará la pista por su código y si existe, se validarán los nuevos datos y se actualizarán sus propiedades. Para la eliminación, se recorrerá la colección mediante índices. Al encontrar la pista, se utilizará `remove(indice)` y se finalizará el recorrido y si la colección está vacía o el código no existe, se informará al usuario sin interrumpir el programa.

10. ¿Qué situaciones del programa pueden producir excepciones? Identifique qué excepciones deberán manejarse y en qué partes del programa utilizará try-catch y finally.

Situación	Excepción o prevención
Ingresar texto o un número no válido cuando se solicita un entero.	Capturar <code>InputMismatchException</code> .
Proporcionar un riesgo, importancia o confiabilidad fuera de los límites permitidos.	Generar y capturar <code>IllegalArgumentException</code> .
Proporcionar textos obligatorios vacíos o datos null no permitidos.	Generar y capturar <code>IllegalArgumentException</code> .
Registrar una pista con código repetido o una ubicación en una posición ocupada.	Generar y capturar <code>IllegalArgumentException</code> .
Intentar modificar o eliminar un registro inexistente.	Generar y capturar <code>IllegalArgumentException</code> .
Indicar una posición fuera del arreglo.	Generar explícitamente <code>IndexOutOfBoundsException</code> al validar la posición y capturarla.
Intentar utilizar una ubicación o pista que contiene null.	Prevenirlo comprobando la referencia antes de utilizar sus métodos.
Solicitar máximos o promedios sin registros.	Comprobar que existan elementos antes de calcular.

Se utilizará try-catch dentro de `leerEntero()` para manejar `InputMismatchException`; al detectar una entrada incorrecta, se limpiará mediante `entrada.nextLine()` y se solicitará nuevamente el número.

También se utilizará try-catch en Principal alrededor de las operaciones del menú para capturar los errores de validación, mostrar un mensaje y permitir que el usuario continúe. El bloque finally se colocará alrededor de la ejecución general del programa para cerrar el Scanner al finalizar, independientemente de que la ejecución termine normalmente o por una excepción.

Diseño

